# Software Requirements Specification

**for**

# Student Assignment Management System

**Prepared by:**

**3. Alishba Khan**

**11. Kalyani Meshram**

**48. Vedant Bhagat**

**Ramdeobaba University**

**14-10-2026**

# Table of Contents

**1. Introduction ..... 1**
1.1 Introduction ..... 1
1.2 Intended Users ..... 1
1.3 Document Overview ..... 1

**2. Use Case Diagram ..... 3**
2.1 Use Case Diagram ..... 3

**3. Activity Diagram ..... 4**
3.1 Overall Activity Diagram ..... 4
3.2 Activity Diagram for Login ..... 5
3.3 Activity Diagram for Submitting Assignment ..... 6
3.4 Activity Diagram for View Assignment ..... 7

**4. Class Diagram ..... 8**
4.1 Class Diagram ..... 8

**5. Sequence Diagram ..... 10**
5.1 Sequence Diagram ..... 10

**6. State Diagram ..... 11**
6.1 State Diagram ..... 11

**7. Deployment Diagram ..... 13**
7.1 Deployment Diagram ..... 13

**8. Component Diagram ..... 14**
8.1 Component Diagram ..... 14

**9. Gantt Chart ..... 15**
9.1 Gantt Chart Diagram ..... 15

**10. Software Requirements Specification Details ..... 16**
10.1 Purpose and Scope ..... 16
10.2 Project Objectives ..... 16
10.3 Functional Requirements ..... 16
10.4 Non-Functional Requirements ..... 18
10.5 External Interface Requirements ..... 18
10.6 Data Requirements ..... 19
10.7 Assumptions and Constraints ..... 19
10.8 Acceptance Criteria ..... 20
10.9 Testing Strategy ..... 20
10.10 Glossary ..... 20

**11. GitHub Link ..... 22**
11.1 GitHub Link ..... 22

# 1. Introduction

## 1.1 Introduction

In most colleges, assignments are still shared through a mix of notice boards, messaging groups, e-mail and printed copies. Students lose track of instructions and due dates, submissions arrive through different channels, and teachers spend a significant amount of time collecting files, checking whether work was turned in on time, and returning marks. Records of who submitted what, and when, are scattered and difficult to verify later.

The **Student Assignment Management System (SAMS)** is a web-based platform that brings the complete assignment lifecycle into one place. Teachers create and publish assignments with instructions, a deadline and maximum marks. Students log in to a personal portal, view the assignments posted for their course, upload their solutions as files and turn them in. The system checks every submission against the deadline, records it with a timestamp, and flags late work automatically.

Teachers then review the submissions from a single dashboard, approve them or ask for a redo, and assign grades with written feedback. Students are notified when a new assignment is published, when a deadline is approaching, and when a grade has been posted. By replacing manual collection with a structured, traceable workflow, the system saves time for teachers, gives students a clear view of their pending and completed work, and maintains a reliable record of every submission and grade.

## 1.2 Intended Users

The system serves two primary user classes, as identified in the use case diagram:

- **Student** – logs in to the student portal, views published assignments and their details, uploads and turns in solutions, tracks submission status, and views grades and feedback.
- **Teacher** – logs in to the teacher portal, creates, updates and publishes assignments, reviews student submissions, requests a redo where required, and assigns grades with feedback.

This document is intended for the project team, the project guide and evaluators, and any developer or tester who will design, build or verify the system.

## 1.3 Document Overview

Sections 2 to 8 present the UML models of the system: use case, activity, class, sequence, state, deployment and component diagrams, each followed by an explanation of what it represents. Section 9 presents the project schedule as a Gantt chart. Section 10 specifies the detailed software requirements, including functional and non-functional requirements, interfaces, data requirements,

assumptions, acceptance criteria and the testing strategy. Section 11 provides the link to the project repository.

# 2. Use Case Diagram

## 2.1 Use Case Diagram

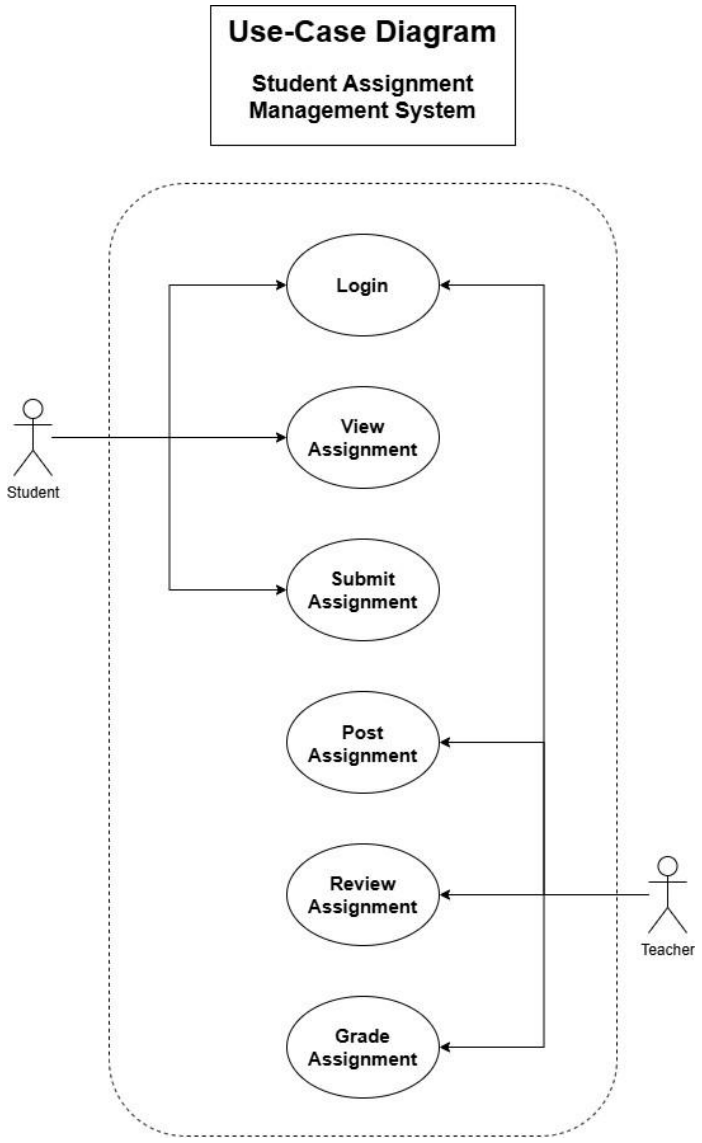



*Figure 2.1: Use Case Diagram of the Student Assignment Management System*

The diagram identifies two actors, Student and Teacher, interacting with six use cases inside the system boundary. Both actors share the **Login** use case, which authenticates the user and opens the correct portal for their role. A Student can **View Assignment** to see the published assignments and their instructions, and **Submit Assignment** to upload and turn in a solution. A Teacher can **Post Assignment** to create and publish new work, **Review Assignment** to examine submitted solutions, and **Grade Assignment** to award marks and feedback. Teacher functions must be available only to authenticated teacher accounts, and a student must only ever see and act on their own submissions.

# 3. Activity Diagram

## 3.1 Overall Activity Diagram

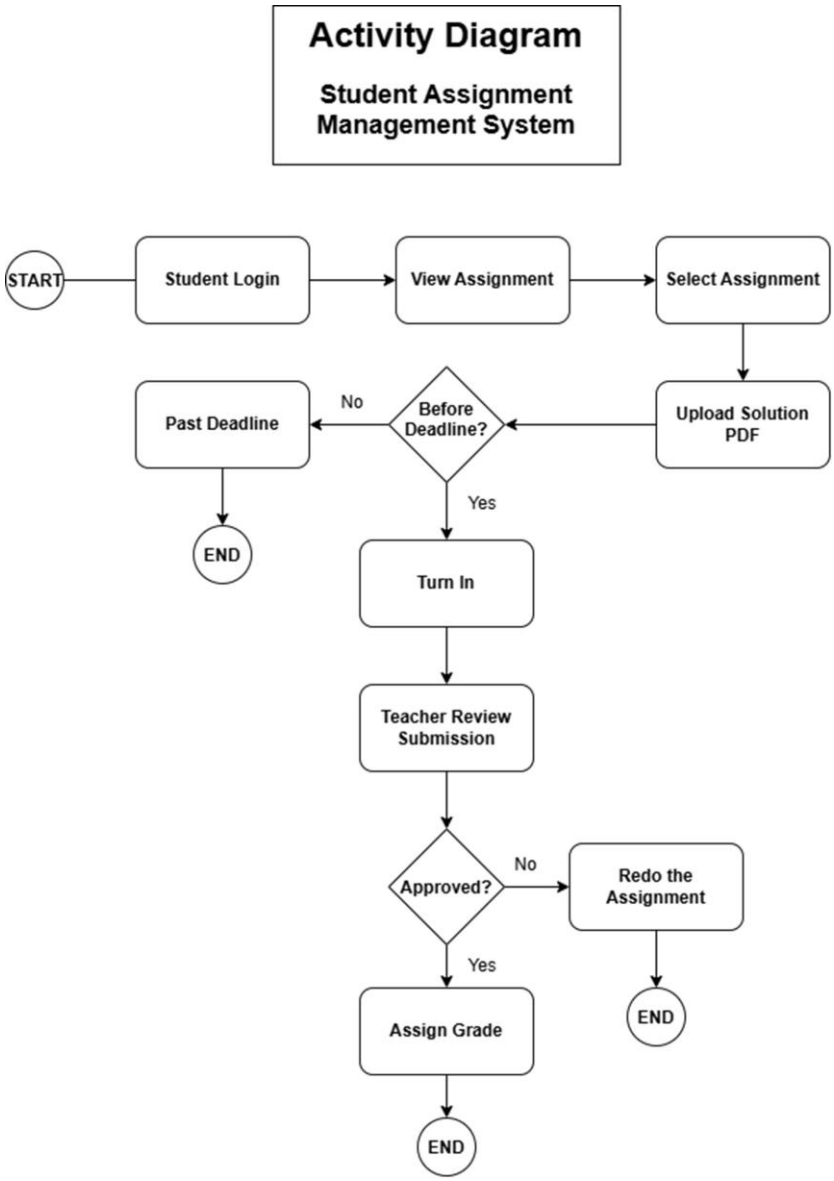



*Figure 3.1: Overall Activity Diagram*

The overall workflow starts when a student logs in, views the list of assignments and selects one. The student uploads a solution PDF and the system checks whether the deadline has passed. If the deadline is still open, the student turns in the work and it moves to the teacher for review. If the deadline has passed and late submissions are not permitted for that assignment, the system shows a "Past Deadline" message and the flow ends. During review, the teacher decides whether the submission is approved. An approved submission is assigned a grade, completing the process; a submission that is not approved is returned to the student to redo the assignment.

## 3.2 Activity Diagram for Login

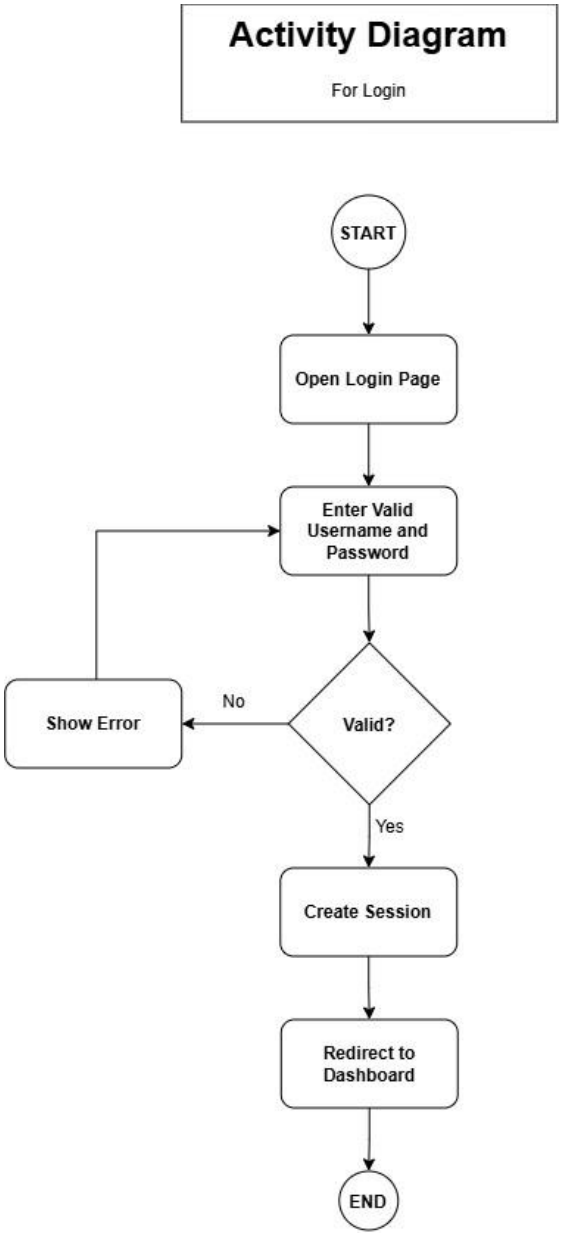



*Figure 3.2: Activity Diagram for Login*

The user opens the login page and enters a username and password. The system validates the credentials against the stored user records. If they are invalid, an error message is shown and the user is returned to the credential entry step to try again; the message should not reveal which of the two fields was incorrect. If they are valid, the system creates a session for the user and redirects them to the dashboard that matches their role.

## 3.3 Activity Diagram for Submitting Assignment

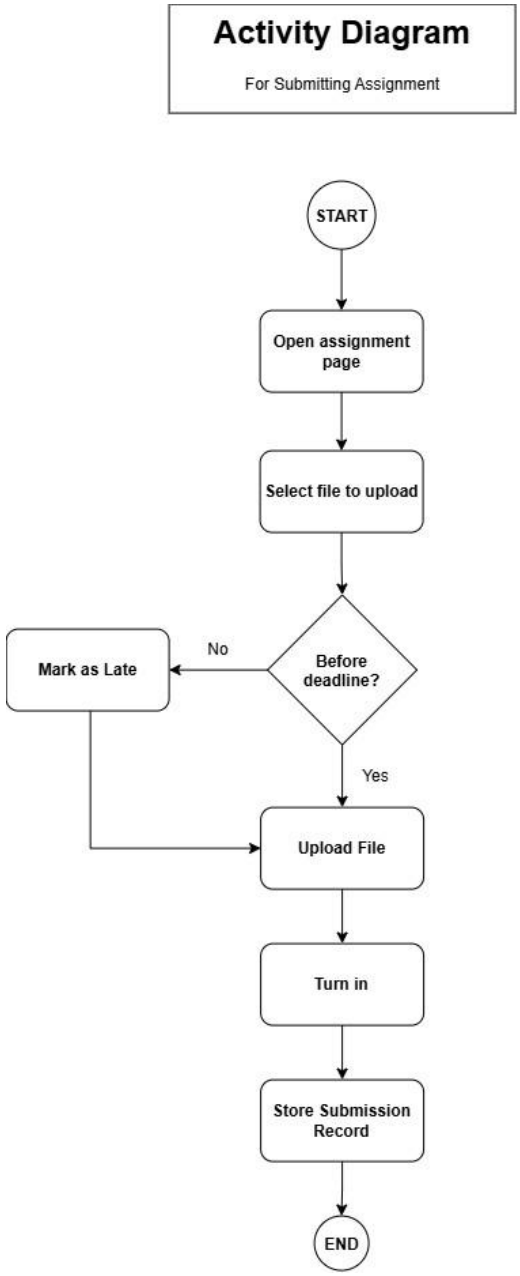



*Figure 3.3: Activity Diagram for Submitting Assignment*

The student opens the assignment page and selects the file to upload. The system checks the submission time against the assignment deadline. A submission made before the deadline proceeds directly to upload; a submission made after the deadline is marked as **Late** and then uploaded, where the teacher allows late work. The student turns in the uploaded file and the system stores the submission record, including the file name, the time of submission and its status. The deadline check must use the server's clock, not the student's device time.

## 3.4 Activity Diagram for View Assignment

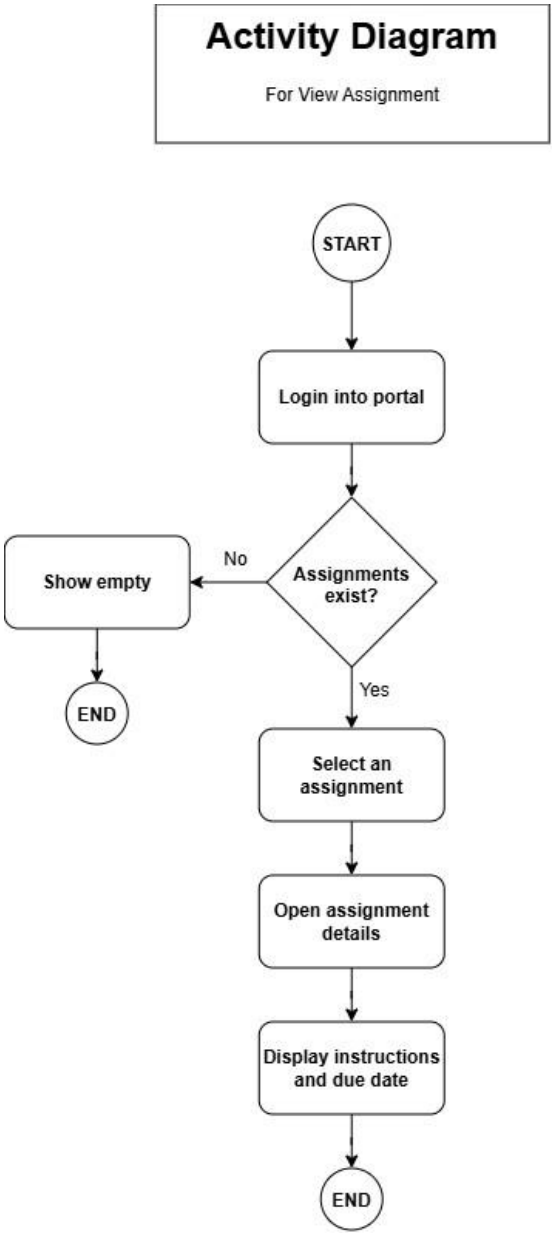



*Figure 3.4: Activity Diagram for View Assignment*

After logging in to the portal, the system checks whether any assignments exist for the student's course. If none exist, an empty-state message is shown instead of a blank page. Otherwise the student selects an assignment and the system opens its details, displaying the instructions and the due date so the student knows exactly what is required and by when.

# 4. Class Diagram

## 4.1 Class Diagram

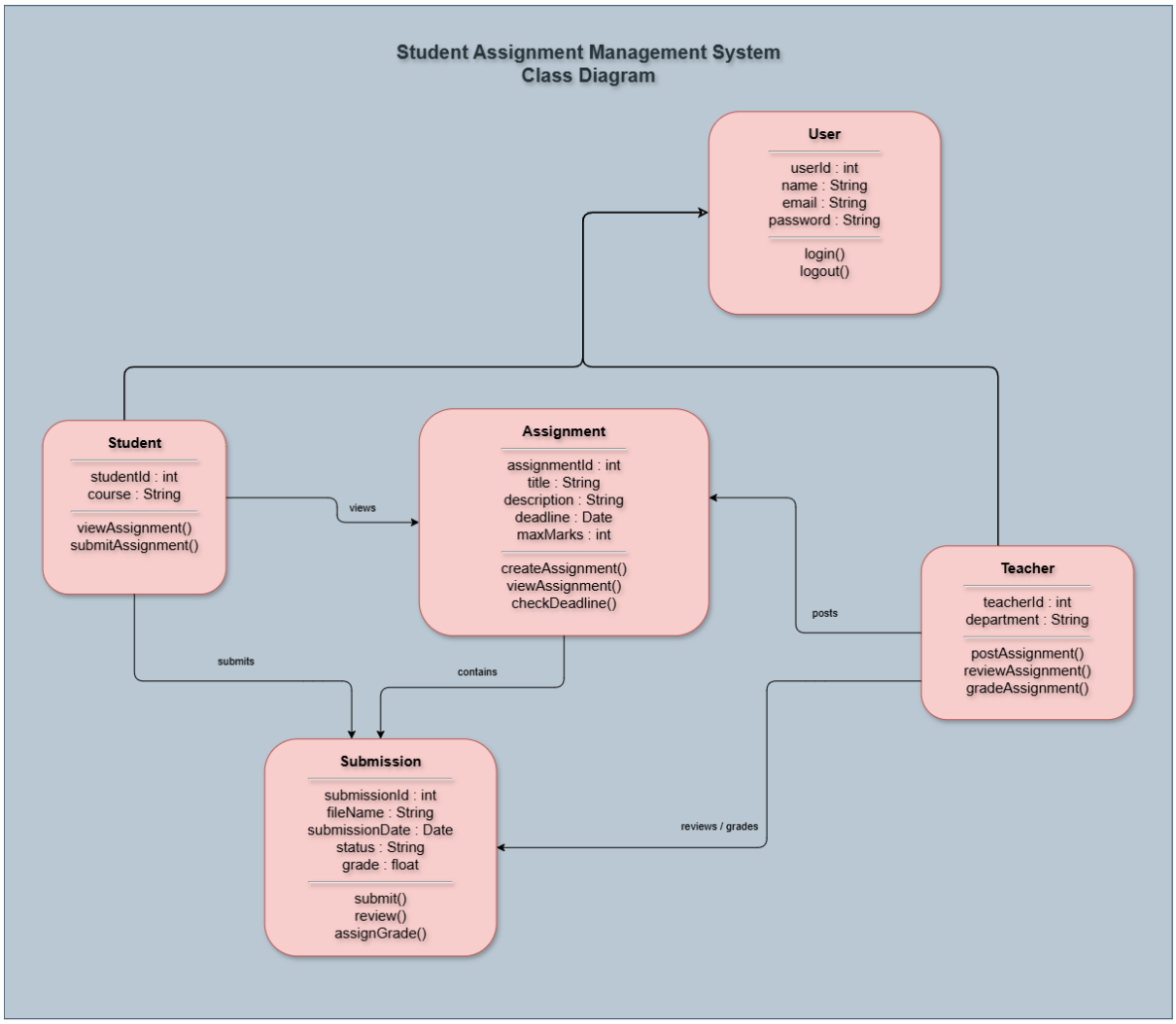



*Figure 4.1: Class Diagram*

The class diagram presents five main classes. **User** holds the common attributes (userId, name, email, password) and the login() and logout() operations. **Student** (studentId, course) and **Teacher** (teacherId, department) both inherit from User, so authentication logic is written once and shared. **Assignment** stores the assignmentId, title, description, deadline and maxMarks, with operations to create and view the assignment and to check the deadline. **Submission** records the submissionId, fileName, submissionDate, status and grade, with operations to submit, review and assign a grade.

The associations show that a Teacher posts Assignments and reviews and grades Submissions, while a Student views Assignments and submits Submissions. Each Assignment contains the Submissions made against it. These classes map directly to the tables in the database and to the

modules of the application. Passwords must never be stored in plain text; the password attribute represents a securely hashed value.

# 5. Sequence Diagram

## 5.1 Sequence Diagram

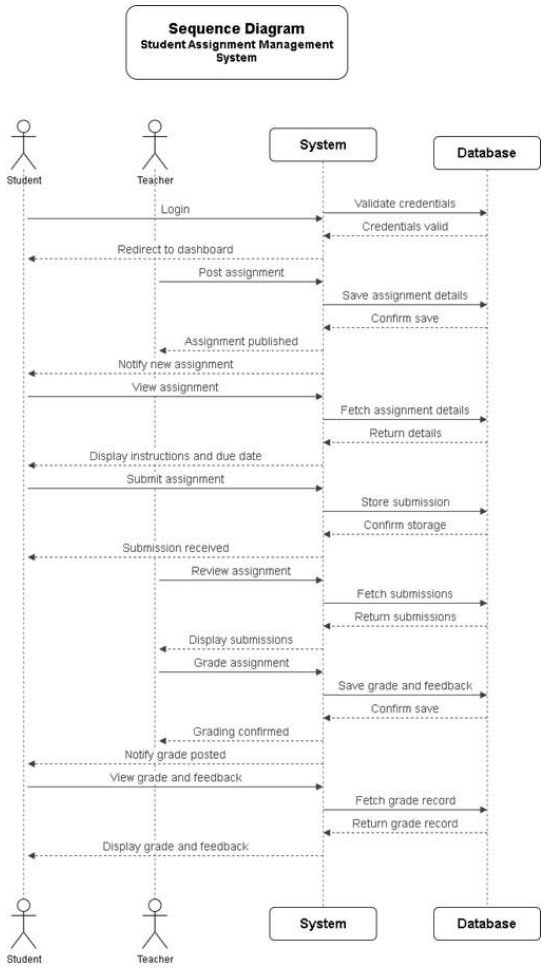



*Figure 5.1: Sequence Diagram*

The sequence diagram shows the messages exchanged between the Student, Teacher, System and Database across one complete assignment cycle. The student logs in and the system validates the credentials with the database before redirecting to the dashboard. The teacher posts an assignment, which the system saves and confirms as published, and the student is notified of the new assignment. The student views the assignment details and submits a solution, which is stored and acknowledged with a "Submission received" response.

The teacher then reviews the submissions fetched from the database and grades them; the grade and feedback are saved and the teacher receives a confirmation. Finally, the student is notified that the grade has been posted and views the grade and feedback. Every write operation is confirmed by the database before the user is told it succeeded, so a user is never shown a success message for data that was not saved.

# 6. State Diagram

## 6.1 State Diagram

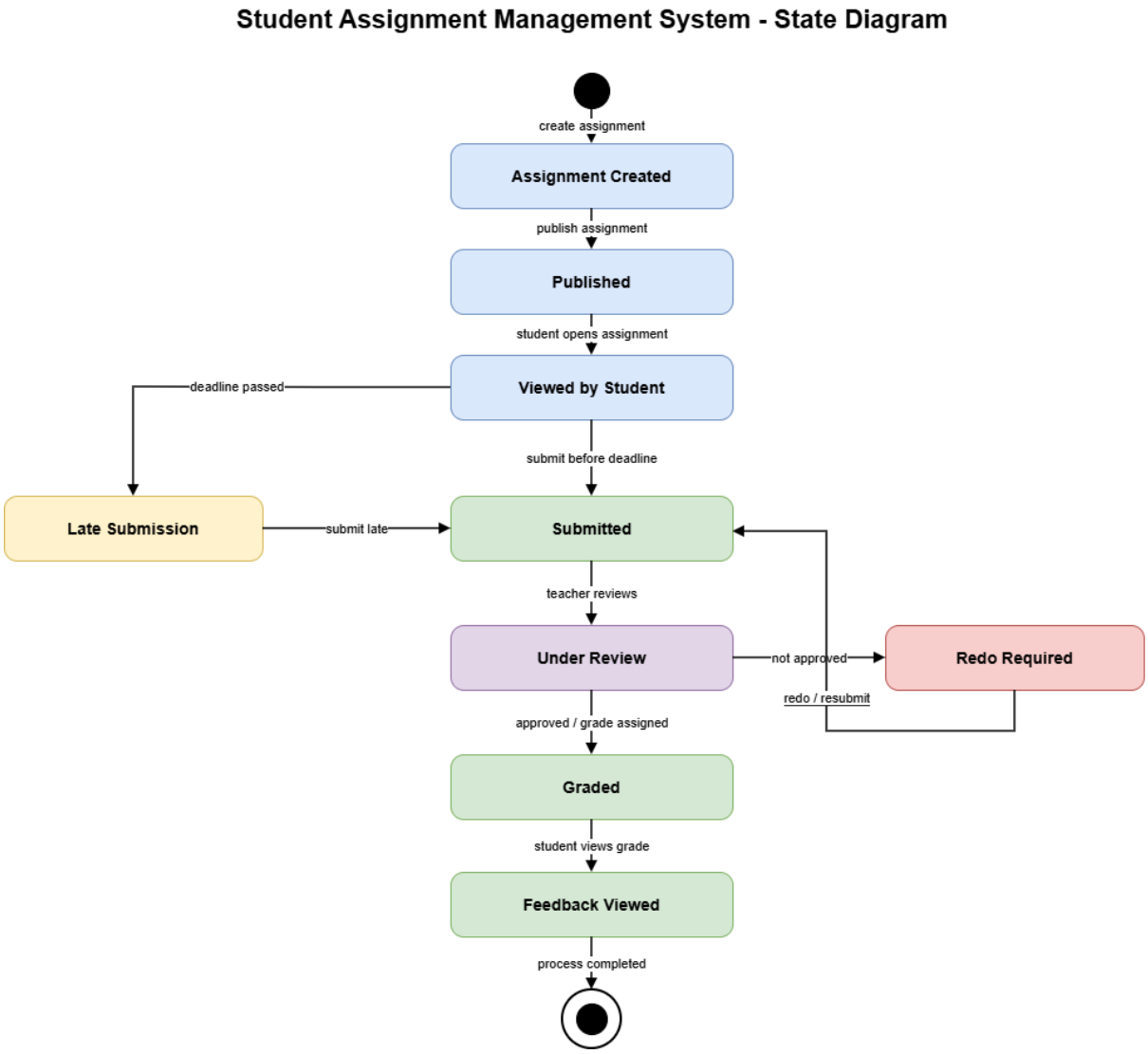



*Figure 6.1: State Diagram*

The state diagram models the life of an assignment and its submission. An assignment begins in the **Assignment Created** state and moves to **Published** once the teacher releases it. When a student opens it, it becomes **Viewed by Student**. A submission made before the deadline moves to **Submitted**; if the deadline has passed, it first passes through **Late Submission** before being submitted, so the late flag is preserved.

When the teacher begins reviewing, the submission is **Under Review**. If it is not approved, it moves to **Redo Required**, and the student's resubmission returns it to Submitted. Once approved and graded, it reaches **Graded**, and after the student views the result it becomes **Feedback Viewed**,

which ends the process. Defining these states explicitly ensures that a grade can only be assigned to a submission that has been reviewed, and that the current status shown to both users is always accurate.

# 7. Deployment Diagram

## 7.1 Deployment Diagram

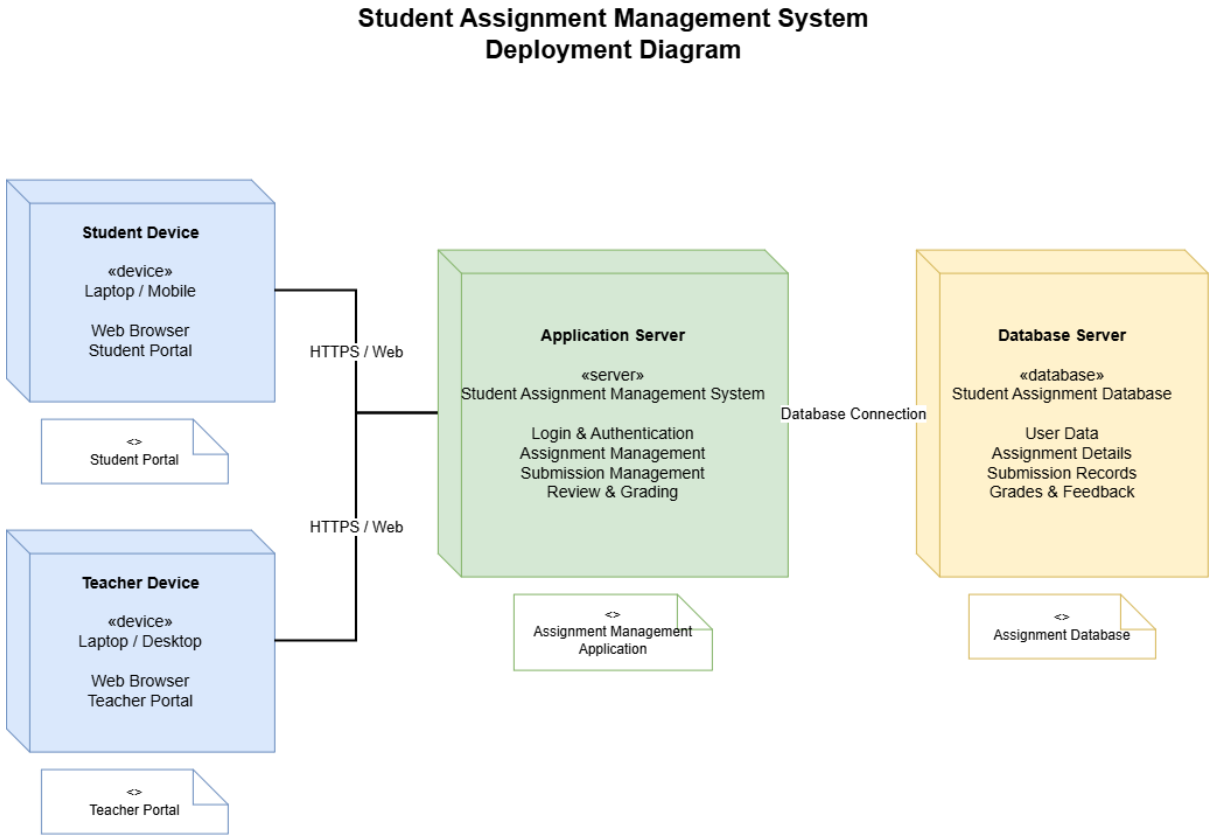



*Figure 7.1: Deployment Diagram*

The deployment diagram shows three types of node. Students use a laptop or mobile device with a web browser to access the **Student Portal**, and teachers use a laptop or desktop browser to access the **Teacher Portal**. Both connect over HTTPS to the **Application Server**, which hosts the Student Assignment Management System and its login and authentication, assignment management, submission management, and review and grading functions.

The application server connects to a separate **Database Server** holding the Student Assignment Database, which stores user data, assignment details, submission records, and grades and feedback. All client traffic must be encrypted, and the database must accept connections only from the application server, never directly from user devices. In a small or local deployment, the application and database may run on the same machine.

# 8. Component Diagram

## 8.1 Component Diagram

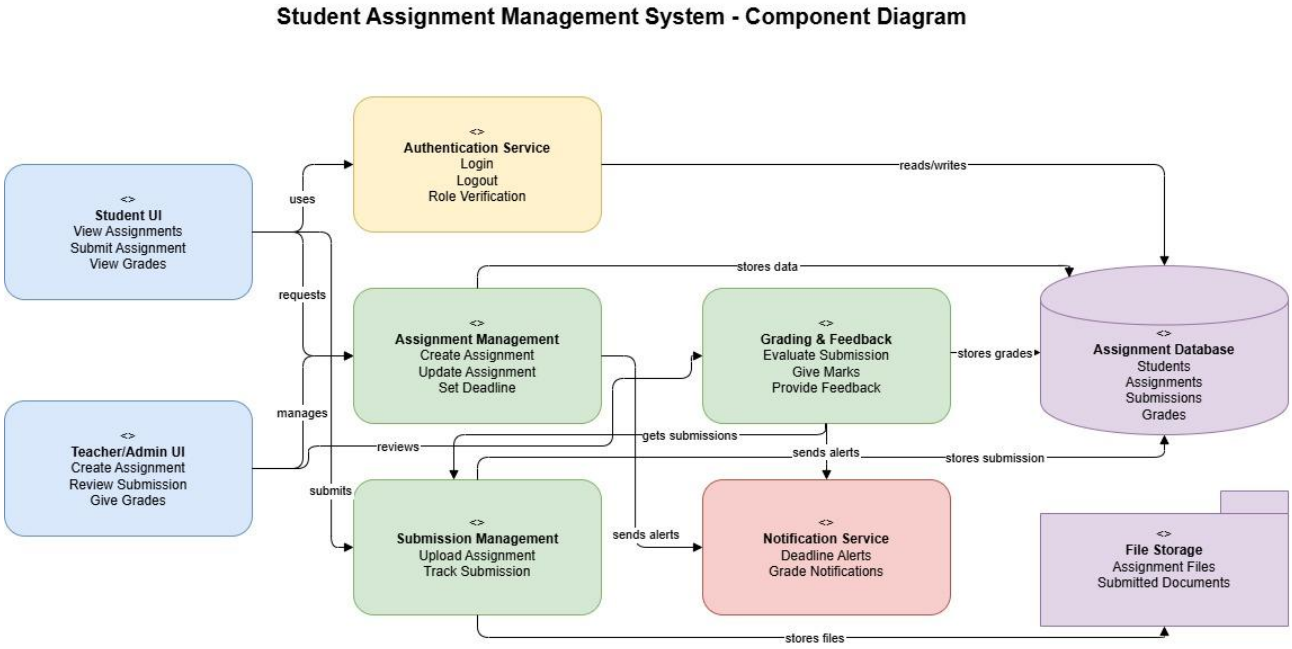



*Figure 8.1: Component Diagram*

The system is divided into well-defined components. The **Student UI** (view assignments, submit assignment, view grades) and the **Teacher/Admin UI** (create assignment, review submission, give grades) use the **Authentication Service** for login, logout and role verification. The **Assignment Management** component creates and updates assignments and sets deadlines. The **Submission Management** component handles uploads and tracks submissions, storing files in **File Storage** and records in the **Assignment Database**.

The **Grading & Feedback** component retrieves submissions, evaluates them, records marks and provides feedback. The **Notification Service** sends deadline alerts and grade notifications when the assignment, submission and grading components raise events. Keeping uploaded files in dedicated file storage, separate from the relational database, keeps the database small and fast. Because the user interfaces communicate only with these services and never with storage directly, each component can be developed, tested and replaced independently.

# 9. Gantt Chart

## 9.1 Gantt Chart Diagram

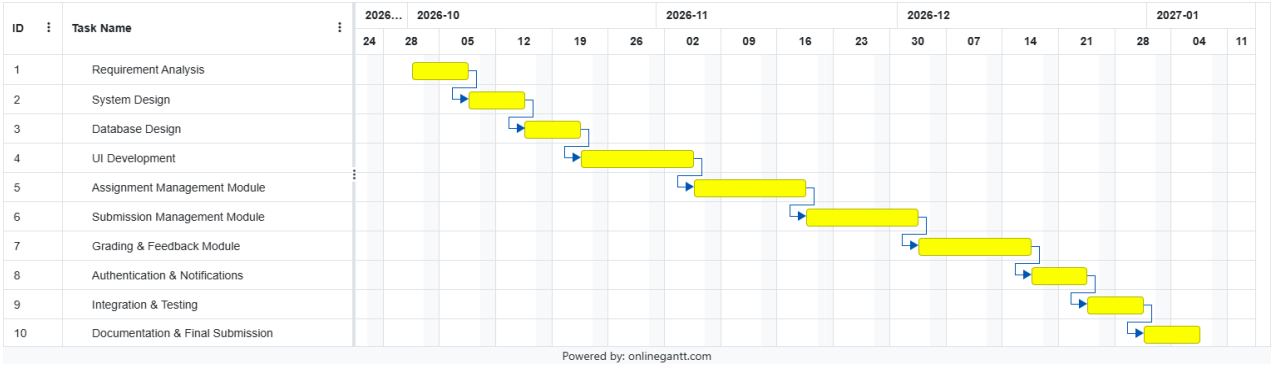



*Figure 9.1: Project Schedule (Gantt Chart)*

The Gantt chart shows the planned sequence of project activities from late September 2026 to early January 2027. The project begins with requirement analysis, followed by system design and database design. UI development then takes place, after which the three core modules are built in turn: the assignment management module, the submission management module, and the grading and feedback module. Authentication and notifications are implemented next, followed by integration and testing, and the project closes with documentation and final submission.

Each task depends on the completion of the previous one, as shown by the dependency arrows. The schedule should be updated whenever a task slips, and the integration and testing phase should be protected, since it is where defects between modules are found and fixed.

# 10. Software Requirements Specification Details

## 10.1 Purpose and Scope

This section specifies the expected behaviour, interfaces, constraints and quality requirements of the Student Assignment Management System. The system allows teachers to create, publish and grade assignments, and allows students to view assignments, submit solutions and receive grades and feedback through a web browser.

**In scope:** user login with role-based access, assignment creation and publishing, assignment viewing, file-based submission with deadline checking and late flagging, review with approval or redo, grading with feedback, notifications, and storage of all records. **Out of scope:** plagiarism detection, online quizzes or examinations, automatic grading of submitted work, attendance management, fee or timetable management, and integration with external learning management systems, unless added in a later version.

## 10.2 Project Objectives

- Provide a single, central platform for publishing, submitting and grading assignments.
- Give students a clear view of their assignments, due dates, submission status, grades and feedback.
- Enforce deadlines automatically and record accurate, server-side submission timestamps.
- Reduce the manual effort teachers spend collecting files and tracking who has submitted.
- Support a review cycle in which teachers can approve work or request a redo.
- Keep users informed through notifications for new assignments, approaching deadlines and posted grades.
- Protect student work and grades through authentication and role-based access control.

## 10.3 Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Allow students and teachers to log in using a registered username (or e-mail) and password. | High |
| FR-02 | Validate credentials against stored user records; on failure, display a generic error message and allow the user to try again. | High |
| FR-03 | On successful login, create a user session and redirect the user to the dashboard for their role (Student or Teacher). | High |
| FR-04 | Enforce role-based access so that only teachers can create, publish, review and grade assignments, and students can access only their own submissions and grades. | High |

| ID | Requirement | Priority |
|---|---|---|
| FR-05 | Allow users to log out, ending the session; sessions shall also expire after a period of inactivity. | Medium |
| FR-06 | Allow a teacher to create an assignment with a title, description/instructions, deadline (date and time) and maximum marks, with an optional reference file. | High |
| FR-07 | Allow a teacher to update an assignment's details, including extending its deadline; changes shall be visible to students immediately. | Medium |
| FR-08 | Allow a teacher to publish an assignment, making it visible to the students of the selected course. | High |
| FR-09 | Allow a teacher to specify, per assignment, whether late submissions are accepted. | Medium |
| FR-10 | Display to a student the list of published assignments for their course, showing title, due date and the student's submission status. | High |
| FR-11 | Display an empty-state message when no assignments exist for the student. | Low |
| FR-12 | Allow a student to open an assignment and view its instructions, due date, maximum marks and any attached reference file. | High |
| FR-13 | Allow a student to select and upload a solution file (PDF) for a selected assignment and turn it in. | High |
| FR-14 | Validate the uploaded file's type and size before accepting it, and reject invalid files with a clear message. | High |
| FR-15 | Check the deadline at the time of turn-in using server time: mark on-time work as Submitted; mark late work as Late where late submissions are allowed; otherwise block the submission with a "Deadline passed" message. | High |
| FR-16 | Store each submission record with the file name, submission date and time, and status, and confirm successful submission to the student. | High |
| FR-17 | Allow a student to view the current status of each of their submissions (Submitted, Late, Under Review, Redo Required, Graded). | Medium |
| FR-18 | Allow a teacher to view all submissions for an assignment, including late flags, and open each submitted file. | High |
| FR-19 | Allow a teacher to approve a submission or mark it Redo Required with comments; a student whose work requires a redo shall be able to resubmit it. | Medium |
| FR-20 | Allow a teacher to assign a grade and written feedback to a reviewed submission; the grade shall be between 0 and the assignment's maximum marks. | High |
| FR-21 | Allow a student to view their grade and feedback once posted, and record that the feedback has been viewed. | High |
| FR-22 | Notify students when a new assignment is published, when a deadline is approaching, when a redo is requested and when a grade is posted. | Medium |

## 10.4 Non-Functional Requirements

| Attribute | Requirement |
|---|---|
| Performance | Under normal load, pages and assignment lists shall load within 3 seconds, and a typical submission file (up to 10 MB) shall upload and be confirmed within a few seconds on a standard broadband connection. |
| Scalability | The system shall handle a spike of concurrent submissions shortly before a deadline without rejecting or losing submissions. |
| Reliability | A submission shall be confirmed to the student only after both the file and its record have been stored successfully; no confirmed submission shall be lost. |
| Availability | The system shall be available during the academic term, with planned maintenance scheduled away from assignment deadlines. |
| Security | Passwords shall be stored as salted hashes, all traffic shall use HTTPS, sessions shall be protected, and access to files and grades shall be checked against the user's role and ownership. |
| Privacy | A student shall never be able to view or download another student's submission or grade. |
| Data integrity | Submission timestamps shall come from the server clock; grades shall be validated against maximum marks; database backups shall be taken regularly. |
| Usability | The interface shall provide clear labels, visible due dates and statuses, helpful validation messages and confirmations, and require no training for basic use. |
| Compatibility | The portals shall work on current versions of common web browsers and adapt to laptop, desktop and mobile screen sizes. |
| Maintainability | Authentication, assignment, submission, grading and notification logic shall be implemented as separate modules, as shown in the component diagram. |

## 10.5 External Interface Requirements

**User interfaces:** the Student Portal shall provide a login page, a dashboard listing assignments with due dates and statuses, an assignment detail page with an upload control and a "Turn In" action, and a page for grades and feedback. The Teacher Portal shall provide a login page, a dashboard of the teacher's assignments, a form to create or edit an assignment, a list of submissions per assignment, and a review screen to approve, request a redo, grade and add feedback. Validation and error messages shall be shown next to the relevant field.

**Hardware interfaces:** users require a laptop, desktop or mobile device capable of running a modern web browser. The server side requires an application server and a database server, or a single machine for a small deployment, with sufficient disk space for uploaded files.

**Software interfaces:** the application server communicates with the relational database for users, assignments, submissions and grades, with file storage for uploaded documents, and with a notification mechanism (in-app and/or e-mail) for alerts. The actual framework, database engine and hosting platform used in the final implementation shall be documented.

**Communication interfaces:** all communication between browsers and the application server shall use HTTPS. The application server shall connect to the database over a secured connection that is not exposed to client devices.

## 10.6 Data Requirements

The system shall store the following data, derived from the class diagram:

| Entity | Key Attributes | Description |
|---|---|---|
| User | userId, name, email, password (hashed), role | Common account details for every user. |
| Student | studentId, course | Student-specific details, linked to a User. |
| Teacher | teacherId, department | Teacher-specific details, linked to a User. |
| Assignment | assignmentId, title, description, deadline, maxMarks, lateAllowed | An assignment posted by a teacher for a course. |
| Submission | submissionId, assignmentId, studentId, fileName, submissionDate, status, grade, feedback | A student's submission for an assignment, with its review and grading result. |

Uploaded files shall be stored in file storage and referenced from the Submission record by a unique path, so that files with the same name from different students do not overwrite each other. Each student shall have at most one active submission per assignment; a resubmission after a redo shall replace or version the earlier file. Records shall be retained for at least the academic term and protected by regular backups.

## 10.7 Assumptions and Constraints

- Student and teacher accounts are created in advance, for example by the institution, and each student is mapped to a course.
- Users have internet access and a modern web browser.
- Submissions are uploaded as PDF files within a defined maximum file size.
- The server clock is the single authoritative time source for all deadline checks.
- Automatic plagiarism checking and automatic grading are outside the scope of this version.
- The project must be completed within the academic schedule shown in the Gantt chart.

- Notification by e-mail depends on the availability of an e-mail service; in-app notifications are the minimum requirement.

## 10.8 Acceptance Criteria

- A valid user can log in and reaches the correct dashboard for their role; invalid credentials display an error without logging the user in.
- A student cannot access teacher functions, and cannot view another student's submissions or grades.
- A teacher can create, publish and edit an assignment, and it appears to the students of the course with the correct details.
- A student can upload a valid PDF and turn it in before the deadline, and it is recorded as Submitted with the correct server timestamp.
- A submission after the deadline is marked Late, or blocked if late submissions are disabled for that assignment.
- Invalid files (wrong type or too large) are rejected with a clear message.
- A teacher can review submissions, request a redo, and assign a grade within the valid range with feedback; the student can then see the grade and feedback.
- Notifications are generated for new assignments, approaching deadlines, redo requests and posted grades.

## 10.9 Testing Strategy

**Unit testing** shall verify individual functions such as credential validation, deadline checking, file validation, status transitions and grade range validation. **Integration testing** shall verify that the user interfaces, authentication service, assignment, submission and grading modules, database and file storage work together correctly, for example that an uploaded file and its record are both saved.

**System testing** shall cover complete end-to-end workflows: login, post assignment, view assignment, submit on time, submit late, redo and resubmit, grade, and view feedback. **Security testing** shall confirm that role-based restrictions cannot be bypassed by changing URLs or request parameters. **Load testing** shall simulate many students submitting just before a deadline. Boundary cases shall include a submission exactly at the deadline, an empty file, a non-PDF file, a file at the maximum size, a grade of zero and a grade equal to the maximum marks.

## 10.10 Glossary

| Term | Meaning |
|---|---|
| SAMS | Student Assignment Management System. |
| Assignment | A task posted by a teacher with instructions, a deadline and maximum marks. |

| Term | Meaning |
|---|---|
| Submission | A file uploaded and turned in by a student in response to an assignment. |
| Turn In | The action that finalises an uploaded file as the student's submission. |
| Deadline | The date and time by which a submission must be turned in to be considered on time. |
| Late Submission | A submission turned in after the deadline, where late work is permitted. |
| Redo Required | A status set by the teacher when a submission must be corrected and resubmitted. |
| Role-based access | Restricting features and data according to whether the user is a Student or a Teacher. |
| Session | The authenticated period between a user's login and logout. |
| HTTPS | Encrypted protocol used for communication between the browser and the server. |

# 11. GitHub Link

## 11.1 GitHub Link

https://github.com/kalyaniii16/Student-Assignment-Management-System